

LCD_20x4 Test-Tools

Zeichentest + Grafiktest fuer HD44780/PCF8574 I2C-LCDs

LCD_20x4 Test-Tools — Übersicht

Zwei eigenständige, projektunabhängige C-Testprogramme für HD44780- kompatible Zeichen-LCDs mit PCF8574-I2C-Backpack. Vollständig dokumentiert direkt im Quellcode (Header-Kommentar am Dateianfang) — diese Seite ist nur die Kurzreferenz. Erfolgreich getestet auf hal-9 mit einem 20x4-Display an `/dev/i2c-1`, Adresse `0x27`.

Beide Tools sind reines C ohne externe Bibliotheken (nur `libc` + Linux-Kernel-I2C-Interface `linux/i2c-dev.h`) und laufen unverändert auf jedem Linux-System mit I2C-Bus — nicht an `hal-7`/`hal-9` gebunden, daher auch in anderen Projekten wiederverwendbar. Bus, I2C-Adresse und Displaygröße sind per Kommandozeilenoption einstellbar (siehe unten), nicht im Code fest verdrahtet.

Dateien

Datei	Zweck
<code>LCD_20x4_test.c</code>	Zeichentest: blättert durch den kompletten druckbaren ASCII-Bereich (Buchstaben, Zahlen, Satz-/Sonderzeichen)
<code>LCD_20x4_graphics_test.c</code>	Grafiktest: eigene CGRAM-Symbole (Herz, Smiley, Pfeil, Haken, Kreuz, Glocke, Note, Gradzeichen) + Sweep durch den erweiterten ROM-Zeichensatz (Codes <code>0x80–0xFF</code>)

Kompilieren

```
gcc -O2 -Wall -Wextra -o LCD_20x4_test LCD_20x4_test.c  
gcc -O2 -Wall -Wextra -o LCD_20x4_gfx LCD_20x4_graphics_test.c
```

Die Namen der ausführbaren Dateien sind bewusst kurz gehalten (13 bzw. 12 Zeichen). Linux führt Prozessnamen nur bis 15 Zeichen — längere Namen lassen sich mit `pkill -x` nicht mehr treffen. Siehe *Endlosschleife starten/stoppen*.

- `-O2` — Optimierungsstufe (Standard, für diese kleinen Tools unkritisch)
- `-Wall -Wextra` — alle gängigen plus zusätzliche Compiler-Warnungen; beide Dateien übersetzen damit **warnungsfrei**
- keine weiteren `-l/-I`-Optionen nötig

Optional zur Absicherung beim Ändern des Codes (findet Speicher-/ Überlauferfehler zur Laufzeit):

```
gcc -O1 -g -Wall -Wextra -fsanitize=address,undefined -o LCD_20x4_test LCD_20x4_test.c
```

Start-Optionen (beide Tools identisch, bis auf -d)

Option	Bedeutung	Standard	Gültig
-b PFAD	I2C-Bus-Geräte-datei	/dev/i2c-1	max. 255 Zeichen
-a HEX	I2C-Adresse des Displays (Hex)	27	03–77
-r N	Zeilenzahl des Displays	4	1–4
-c N	Spaltenzahl des Displays	20	1–40
-d SEK	Anzeigedauer je Seite (nur LCD_20x4_test)	4	0–3600
-1	Nur einmal durchlaufen statt Endlosschleife	aus	—
-h	Kurzhilfe anzeigen	—	—

Alle Zahlenwerte werden streng geprüft: nicht numerische Angaben (z. B. -r 4x) und Werte außerhalb des gültigen Bereichs führen zu einer Fehlermeldung und Exit-Code 1, statt still auf einen Ersatzwert zurückzufallen.

Beispiele:

```
./LCD_20x4_test           # Standardwerte, Endlosschleife
./LCD_20x4_test -a 3f -1  # andere I2C-Adresse, nur 1x durchlaufen
./LCD_20x4_test -r 2 -c 16 -d 3  # z. B. für ein 16x2-Display
./LCD_20x4_gfx -1         # Grafiktest, ein Durchlauf
```

Die Adresse vorab mit `i2cdetect -y 1` ermitteln, falls unklar (`i2c-tools` muss installiert sein: `sudo apt install -y i2c-tools`; Benutzer sollte in der Gruppe `i2c` sein: `sudo usermod -aG i2c $USER`).

Exit-Codes

Code	Bedeutung
0	reguläres Ende (auch nach Strg+C/SIGTERM)
1	ungültige Kommandozeilenoption oder Gerät nicht zu öffnen
2	Abbruch wegen wiederholter I2C-Schreibfehler (falsche Adresse oder Display nicht angeschlossen)

Code 2 ist praktisch, um in einem Skript zu erkennen, dass das Display gar nicht antwortet — die Programme brechen dann nach 20 aufeinander- folgenden Fehlern ab, statt endlos Fehlermeldungen zu produzieren.

Endlosschleife starten / stoppen

Vordergrund (Ausgabe direkt sichtbar, mit `Strg+C` beenden — Display wird dabei sauber gelöscht, kein "eingefrorenes" Bild):

```
./LCD_20x4_test
^C
```

Hintergrund (läuft über das Ende der SSH-Sitzung hinaus weiter):

```
nohup ./LCD_20x4_test >lcd_test.log 2>&1 &
echo $!                # PID merken, um es später zu stoppen
```

Stoppen:

```
kill <PID>                # zuverlässigste Variante, Display wird gelöscht
# falls PID nicht mehr bekannt:
pkill -x LCD_20x4_test
pkill -x LCD_20x4_gfx
```

Beide Programme fangen `SIGINT` (`Strg+C`) und `SIGTERM` (`kill/pkill`) ab und löschen das Display sauber, bevor sie sich beenden — kein hartes `kill -9` nötig. Die Reaktion erfolgt innerhalb von ca. 0,1 s, auch mitten in einer langen Seitenanzeige (gemessen: 0,09 s bei `-d 60`).

Fallstricke beim Beenden über den Namen

`pkill -f` **nicht verwenden**. `-f` durchsucht die komplette Kommandozeile und trifft damit auch jede Shell bzw. jedes Skript, in deren Kommandozeile der *Startbefehl* vorkommt. Ein Skript, das das Programm startet und später wieder beenden will, beendet damit sich selbst — über SSH reißt die Sitzung ab. Auch verankerte Muster (`'/NAME$'`, `'(^|/)NAME( |$)'`) helfen nicht zuverlässig, weil der Startbefehl selbst auf das Muster passt. Praktisch getestet:

Variante	LCD_20x4_test	LCD_20x4_gfx	Nebenwirkung
<code>kill <PID></code>	✓	✓	keine
<code>pkill -x NAME</code>	✓	✓	keine
<code>pkill -f NAME</code>	✓	✓	killt die aufrufende Shell/Sitzung
<code>pkill -f '/NAME\$'</code>	✗ (mit Argumenten)	✗ (mit Argumenten)	—

`pgrep -f` **täuscht**. `pgrep -f LCD_20x4_test` meldet scheinbar noch einen laufenden Prozess, obwohl keiner aktiv ist — gefunden wird die eigene Shell, deren Kommandozeile den Suchbegriff enthält. Zum Prüfen daher `pgrep -x` verwenden:

```
pgrep -x LCD_20x4_test      # läuft eine Instanz?
sudo fuser -v /dev/i2c-1    # wer haelte den I2C-Bus offen?
```

Der zweite Befehl ist der verlässlichste Test überhaupt: Er zeigt unabhängig vom Prozessnamen jeden Prozess, der das Display belegt.

Für dauerhaften Betrieb (nicht nur zum Testen)

Diese beiden Tools sind bewusst als **Testprogramme** gedacht (Start per Hand, `nohup` für längere Läufe). Für einen dauerhaft laufenden Status-Dienst (wie das spätere PostgreSQL/Patroni-Statusdisplay) eignet sich stattdessen eine `systemd`-Unit nach dem Vorbild von `hal-lcd.service` aus [lcd-status-hal7.md](#).

Robustheit / geprüfte Schwachstellen

Beide Dateien wurden auf Programmierfehler und Robustheitsprobleme durchgesehen und überarbeitet. Behoben wurden unter anderem:

Problem	Auswirkung	Lösung
Off-by-one: Stringlänge 16 für einen 15-Zeichen-Text	zeigte ein undefiniertes Zusatzzeichen an	Längen kommen jetzt durchgängig aus <code>lcd_print_str()</code> statt aus handgezählten Konstanten
-d unbegrenzt (z. B. -d 999999999)	Integer-Überlauf in der Warteschleife (undefiniertes Verhalten)	Wertebereich 0–3600 erzwungen, Rechnung in <code>long</code>
Zeilenoffsets fest für 20 Spalten verdrahtet	falsche Zeilenadressen bei 4-zeiligen Displays anderer Breite (z. B. 16x4)	Offsets werden aus der Spaltenzahl berechnet
I2C-Adresse ungeprüft	reservierte/ungültige Adressen wurden angenommen	Bereich 03–77 erzwungen
<code>atoi()</code> ohne Fehlererkennung	Tippfehler wie <code>-r 4x</code> wurden still zu einem Ersatzwert	strenge <code>strtol()</code> -Auswertung, Abbruch mit Exit-Code 1
Dauerhafte I2C-Fehler	endlose Fehlerausgabe, Programm lief weiter	Abbruch nach 20 Fehlern in Folge, Exit-Code 2
<code>len</code> -Parameter ohne untere Schranke	negativer Wert hätte zu einer riesigen <code>memcpy</code> -Länge geführt	gegen 0 und Displaybreite geklemmt; Puffergröße an <code>MAX_COLS</code> gekoppelt
<code>signal()</code> mit <code>SA_RESTART</code>	Abbruch wirkte erst nach Ende der Seitenanzeige	<code>sigaction()</code> ohne <code>SA_RESTART</code> , Reaktion in ~0,1 s
CGRAM-Adresszähler blieb nach <code>lcd_create_char()</code> stehen	Folgeausgaben hätten ins CGRAM statt aufs Display geschrieben	Funktion setzt am Ende wieder eine DDRAM-Adresse

Verifiziert auf hal-9: warnungsfrei mit `-Wall -Wextra`, keine Funde unter Address-/UB-Sanitizer in fünf Displaygeometrien (1x1, 2x16, 3x20, 4x20, 4x40), Argumentprüfung mit 11 ungültigen und 5 gültigen Grenzwerten getestet.

Eigene CGRAM-Symbole anpassen

In `LCD_20x4_graphics_test.c` stehen die 8 Beispiel-Icons als einfache 8-Byte-Arrays (`CHAR_HEART`, `CHAR_SMILEY`, ...). Jedes Byte ist eine Pixelzeile des 5x8-Zeichens (untere 5 Bit zählen). Eigene Symbole lassen sich z. B. mit einem der gängigen Online-"HD44780 Custom Character Generator"-Tools entwerfen und als weiteres 8-Byte-Array eintragen (maximal 8 Symbole gleichzeitig, der Controller hat nur 8 CGRAM-Slots).